

N-Body Simulation

Richard B. Anderson

December 2024

Contents

1	Störmer-Verlet Integrator	2
2	N-Body Hamiltonian	3
3	Binary System Simulation: $O(n^2)$	4
4	Tree Solvers	6
4.1	Quad Tree	6
4.2	Multipole Expansion	8
5	Fast Multipole Method: $O(n \log(n)) - O(n)$	11
6	Multipole-QuadTree N-Body Simulation	21

1 Störmer-Verlet Integrator

We wish to use the Hamiltonian $H(p, q)$ in order to derive a numerical algorithm for integrating the motion of a particle over time. We consider hamiltons equations:

$$\dot{q} = \frac{\partial H}{\partial p} \quad \dot{p} = -\frac{\partial H}{\partial q}$$

So now, we will consider how we can step forward the system. We start with a half step in momentum:

$$\begin{aligned} p_{n+1/2} &= p_n + \frac{h}{2} \dot{p}(q_n) \\ q_{n+1} &= q_n + h \dot{q}(p_{n+1/2}) \\ p_{n+1} &= p_{n+1/2} + \frac{h}{2} \dot{p}(q_{n+1}) \end{aligned}$$

And from Hamiltons Equations:

$$\begin{aligned} p_{n+1/2} &= p_n - \frac{h}{2} \frac{\partial H}{\partial q}(q_n) \\ q_{n+1} &= q_n + h \frac{\partial H}{\partial p}(p_{n+1/2}) \\ p_{n+1} &= p_{n+1/2} - \frac{h}{2} \frac{\partial H}{\partial q}(q_{n+1}) \end{aligned}$$

Hamiltonian Stormer-Verlet

Input: Initial position q_0 , initial momentum p_0 , time step h , number of steps N

Output: Arrays q, p containing positions and momenta over time

```

 $q[0] \leftarrow q_0$ 
 $p[0] \leftarrow p_0$ 
for  $n = 0$  to  $N - 1$  do
     $p_{n+\frac{1}{2}} \leftarrow p[n] - \frac{h}{2} \frac{\partial H}{\partial q}(q[n])$  ▷ Half-step momentum update at  $t = n \cdot h$ 
     $q[n+1] \leftarrow q[n] + h \frac{\partial H}{\partial p}(p_{n+\frac{1}{2}})$  ▷ Full-step position update using half-step momentum
     $p[n+1] \leftarrow p_{n+\frac{1}{2}} - \frac{h}{2} \frac{\partial H}{\partial q}(q[n+1])$  ▷ Final half-step momentum update at  $t = (n+1) \cdot h$ 
end for

```

2 N-Body Hamiltonian

For N bodies, due to the superposition principal, we can write the kinetic energy as:

$$T = \frac{1}{2} \sum_{i=1}^N \frac{\mathbf{p}_i \cdot \mathbf{p}_i}{m_i}$$

And the potential energy as:

$$V = G \sum_{i < j} \frac{m_i m_j}{|\mathbf{q}_i - \mathbf{q}_j|}$$

Therefore, assuming a conservative system, we can write the Hamiltonian as the total energy of the system:

$$H = \frac{1}{2} \sum_{i=1}^N \frac{\mathbf{p}_i \cdot \mathbf{p}_i}{m_i} + G \sum_{i < j} \frac{m_i m_j}{|\mathbf{q}_i - \mathbf{q}_j|}$$

From Hamilton's equations, we can write:

$$\dot{\mathbf{q}}_i = \frac{\partial H}{\partial \mathbf{p}_i} = \frac{\mathbf{p}_i}{m_i} \quad \dot{\mathbf{p}}_i = -\frac{\partial H}{\partial \mathbf{q}_i} = -G \sum_{i \neq j} \frac{m_i m_j (\mathbf{q}_i - \mathbf{q}_j)}{|\mathbf{q}_i - \mathbf{q}_j|^3}$$

So, now recalling the protocol for Stormer-Verlet integration:

$$\begin{aligned} p_{n+1/2} &= p_n - \frac{h}{2} \frac{\partial H}{\partial q}(q_n) \\ q_{n+1} &= q_n + h \frac{\partial H}{\partial p}(p_{n+1/2}) \\ p_{n+1} &= p_{n+1/2} - \frac{h}{2} \frac{\partial H}{\partial q}(q_{n+1}) \end{aligned}$$

Now, subbing in our $\dot{\mathbf{q}}_i$'s and $\dot{\mathbf{p}}_i$'s :

$$\begin{aligned} \mathbf{p}_{n+1/2}^i &= \mathbf{p}_n^i - \frac{hG}{2} \sum_{i \neq j} \frac{m^i m^j (\mathbf{q}_n^i - \mathbf{q}_n^j)}{|\mathbf{q}_n^i - \mathbf{q}_n^j|^3} \\ \mathbf{q}_{n+1}^i &= \mathbf{q}_n^i + \frac{h \mathbf{p}_{n+1/2}^i}{m_i} \\ \mathbf{p}_{n+1}^i &= \mathbf{p}_{n+1/2}^i - \frac{hG}{2} \sum_{i \neq j} \frac{m^i m^j (\mathbf{q}_{n+1}^i - \mathbf{q}_{n+1}^j)}{|\mathbf{q}_{n+1}^i - \mathbf{q}_{n+1}^j|^3} \end{aligned}$$

N-Body Stormer-Verlet

Input: Initial positions $\mathbf{q}[0]$, initial momenta $\mathbf{p}[0]$, timestep h , number of steps N

Output: Arrays $\mathbf{q}$, $\mathbf{p}$ with positions and momenta over time

for $n = 0$ to $N - 1$ **do**

▷ Half-step momentum update

for $i = 1$ to N **do**

$$\mathbf{p}_{i,n+\frac{1}{2}} = \mathbf{p}_{i,n} - \frac{hG}{2} \sum_{j \neq i} \frac{m_i m_j (\mathbf{q}_{i,n} - \mathbf{q}_{j,n})}{|\mathbf{q}_{i,n} - \mathbf{q}_{j,n}|^3}$$

end for

▷ Full-step position update

for $i = 1$ to N **do**

$$\mathbf{q}_{i,n+1} = \mathbf{q}_{i,n} + h \frac{\mathbf{p}_{i,n+\frac{1}{2}}}{m_i}$$

end for

▷ Final half-step momentum update

for $i = 1$ to N **do**

$$\mathbf{p}_{i,n+1} = \mathbf{p}_{i,n+\frac{1}{2}} - \frac{hG}{2} \sum_{j \neq i} \frac{m_i m_j (\mathbf{q}_{i,n+1} - \mathbf{q}_{j,n+1})}{|\mathbf{q}_{i,n+1} - \mathbf{q}_{j,n+1}|^3}$$

end for

end for

3 Binary System Simulation: $O(n^2)$

Using this scheme, I ran a simulation with two central masses of $1 M_\odot$ each, and the mass of the orbiting bodies are given by a uniform random variable $10^{-6} \text{ kg} \leq m \leq 10^{15} \text{ kg}$. They are initialized with their initial Keplerian velocity, plus some small perturbation:

$$v_0 = \epsilon + \sqrt{\frac{GM}{r_0}}$$

Where, in my simulation, I set the time-step equal to the dynamical time :

$$\Delta t = t_{dyn} = \sqrt{\frac{r_0^3}{GM}}$$

I ran this simulation for five years, with the orbits mainly remaining stable around 1 AU as the simulation progresses. Here I show the radial positions of the particles, with the color of each line corresponding to the mass of each particle.

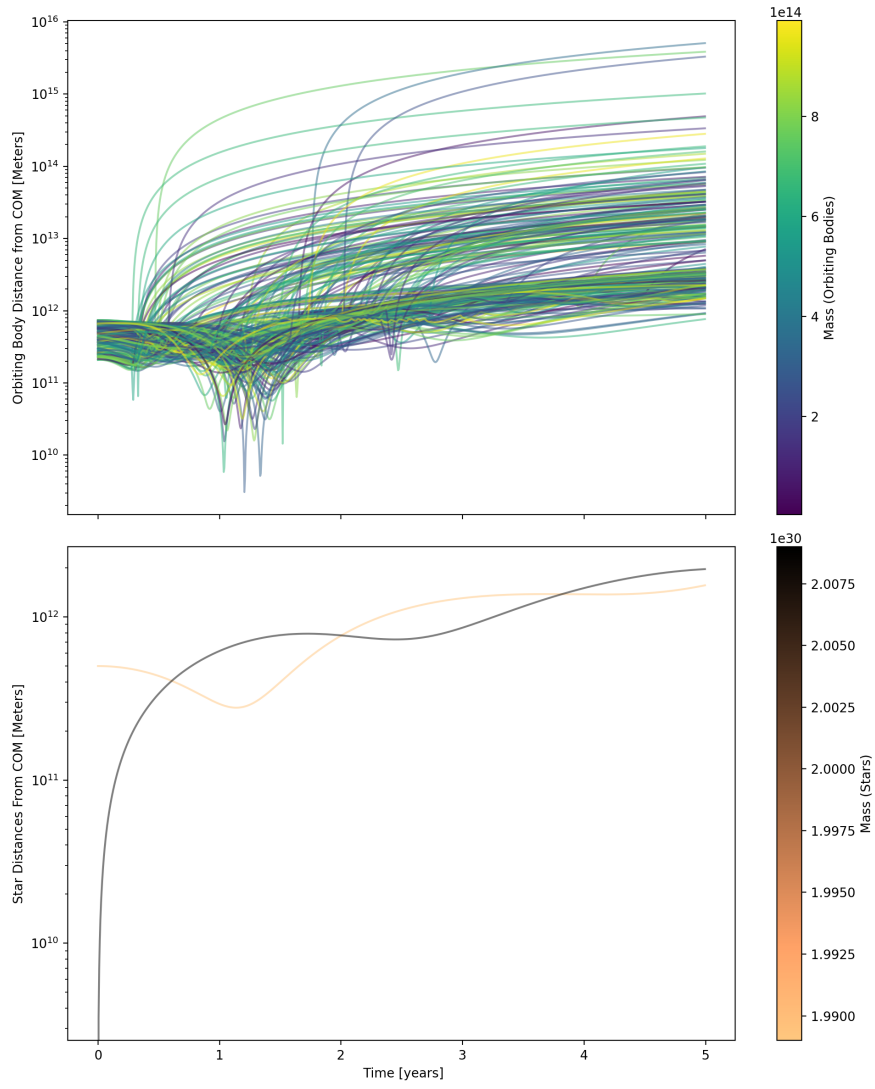


Figure 1: Orbiting Body Positions (Top) and Star Positions (Bottom)

4 Tree Solvers

4.1 Quad Tree

The Barnes-Hut algorithm uses a quadtree to organize data in two dimensions. The quadtree starts with a cell defined by the minimum and maximum bounds of the data. This cell is subdivided into four quadrants using the midpoints of its bounds.

If a quadrant contains fewer than a specified threshold of points, $N_{\max}$, it is not subdivided further. Otherwise, the quadrant is recursively divided into four child cells until all cells meet the threshold or a maximum number of iterations is reached.

The procedure can be summarized as follows:

QuadTree Construction

- 1: **Input:** (Data, Minimum Position, Maximum Position, Maximum Iterations)
- 2: **Output:** QuadTree Structure
- 3: **function** QUADTREE(Data, $x_{\min}$, $y_{\min}$, $x_{\max}$, $y_{\max}$, Maximum Iterations)
- 4: **if** Maximum Iterations = 0 **or** Data is empty **then**
- 5: **return** None
- 6: **end if**
- 7: $x_{\text{mid}} \leftarrow \frac{1}{2}[x_{\min} + x_{\max}]$
- 8: $y_{\text{mid}} \leftarrow \frac{1}{2}[y_{\min} + y_{\max}]$
- 9: **Subdivisions**₁ $\leftarrow$ Data points where $x < x_{\text{mid}}$ **and** $y < y_{\text{mid}}$
- 10: **Subdivisions**₂ $\leftarrow$ Data points where $x < x_{\text{mid}}$ **and** $y \geq y_{\text{mid}}$
- 11: **Subdivisions**₃ $\leftarrow$ Data points where $x \geq x_{\text{mid}}$ **and** $y < y_{\text{mid}}$
- 12: **Subdivisions**₄ $\leftarrow$ Data points where $x \geq x_{\text{mid}}$ **and** $y \geq y_{\text{mid}}$
- 13: **Children** $\leftarrow []$
- 14: **if** **Subdivisions**₁ is not empty **then**
- 15: Add QUADTREE(**Subdivisions**₁, $x_{\min}$, $y_{\min}$, x_{mid} , y_{mid} , Maximum Iterations - 1) to **Children**
- 16: **end if**
- 17: **if** **Subdivisions**₂ is not empty **then**
- 18: Add QUADTREE(**Subdivisions**₂, $x_{\min}$, y_{mid} , x_{mid} , $y_{\max}$, Maximum Iterations - 1) to **Children**
- 19: **end if**
- 20: **if** **Subdivisions**₃ is not empty **then**
- 21: Add QUADTREE(**Subdivisions**₃, x_{mid} , $y_{\min}$, $x_{\max}$, y_{mid} , Maximum Iterations - 1) to **Children**
- 22: **end if**
- 23: **if** **Subdivisions**₄ is not empty **then**
- 24: Add QUADTREE(**Subdivisions**₄, x_{mid} , y_{mid} , $x_{\max}$, $y_{\max}$, Maximum Iterations - 1) to **Children**
- 25: **end if**
- 26: **return** **Children**

```

27: end function
28: function INITIALIZEQUADTREE(Data, Maximum Iterations)
29:    $x_{\min} \leftarrow \min_x(\text{Data})$ 
30:    $y_{\min} \leftarrow \min_y(\text{Data})$ 
31:    $x_{\max} \leftarrow \max_x(\text{Data})$ 
32:    $y_{\max} \leftarrow \max_y(\text{Data})$ 
33:   return QUADTREE(Data,  $x_{\min}$ ,  $y_{\min}$ ,  $x_{\max}$ ,  $y_{\max}$ , Maximum Iterations)
34: end function

```

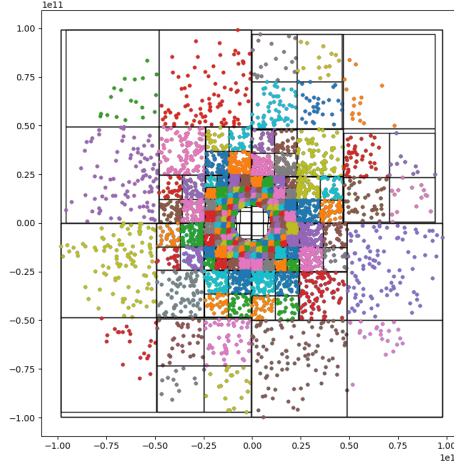


Figure 2: Example Quadtree

Here we can see that the quad-tree divides denser regions into smaller partitions, and less dense regions into larger ones, ensuring that the multi-pole approximation is valid at these points. This tree structure is particularly useful in N -Body solvers because it reduces the number of gravitational potential calculations needed when integrating motion. A key parameter, θ , determines whether a cell can be approximated as a single particle.

To calculate θ , we first compute the size of the cell $\mathcal{C}$ as:

$$S_{\mathcal{C}} = \max_{i \in \mathcal{C}} |x_i - \bar{x}_{\mathcal{C}}|$$

where $\bar{x}_{\mathcal{C}}$ is the center of mass of the cell, given by:

$$\bar{x}_{\mathcal{C}} = \frac{\sum_{i \in \mathcal{C}} m_i x_i}{\sum_{i \in \mathcal{C}} m_i}$$

The parameter θ for a particle at position x_j is then:

$$\theta_j = \frac{S_c}{|x_j - \bar{x}_c|}$$

By setting a maximum threshold $\theta_{\max}$, we can decide whether to approximate the particles in a given cell as one. Specifically, if $\theta_j < \theta_{\max}$, the cell can be approximated as a single particle when calculating the potential at (x_j, y_j) .

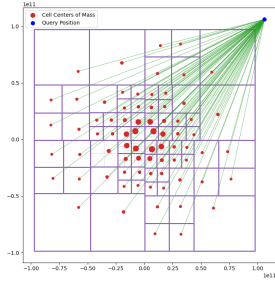


Figure 3: QuadTree for body $atr = 1 AU$

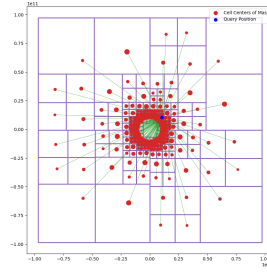


Figure 4: QuadTree for body at $r = 0.1 AU$

4.2 Multipole Expansion

Now for actually calculating this potential, we can write the potential at some point x_j due to some cell $\mathcal{C}$ as a function of the separation of that particle from the particles in the cell :

$$\varphi(x_j) = G \sum_{i \in \mathcal{C}} m_i S(|x_j - x_i|)$$

We will want to Taylor expand $S(|x_j - x_i|)$, but to do this efficiently, we will introduce *multi-index* notation. In N -dimensional space, a multi-index $\mathbf{n}$ is defined as an N -dimensional tuple of non-negative integers:

$$\mathbf{n} = (n_1, n_2, \dots, n_N)$$

where each $n_i \geq 0$ corresponds to the power of the i -th coordinate.

Using this notation, we define:

- The *magnitude* of $\mathbf{n}$ as:

$$|\mathbf{n}| = \sum_{i=1}^N n_i$$

which represents the total degree of the term.

- The *factorial* of $\mathbf{n}$ as:

$$\mathbf{n}! = n_1! \cdot n_2! \cdot \dots \cdot n_N!$$

used for normalizing higher-order terms.

- A product of coordinates raised to $\mathbf{n}$ as:

$$\mathbf{x}^{\mathbf{n}} = x_1^{n_1} x_2^{n_2} \dots x_N^{n_N}$$

- The *gradient* with respect to $\mathbf{x}$ raised to $\mathbf{n}$ is given by:

$$\nabla^{\mathbf{n}} = \frac{\partial^{|\mathbf{n}|}}{\partial x_1^{n_1} \partial x_2^{n_2} \dots \partial x_N^{n_N}}$$

- The binomial coefficient is given as :

$$\binom{\mathbf{n}}{\mathbf{k}} = \prod_{i=1}^d \binom{n_i}{k_i}$$

where $\nabla^{\mathbf{n}}$ denotes the partial derivative operator corresponding to the multi-index $\mathbf{n}$.

So, we can write the Taylor Expansion of a multi variable function about some tuple $\mathbf{a}$ as :

$$f(\mathbf{x}) \approx \sum_{|\mathbf{n}| < p} \frac{\nabla^{\mathbf{n}} f(\mathbf{a})}{\mathbf{n}!} (\mathbf{x} - \mathbf{a})^{\mathbf{n}}$$

Where p is the maximum order of the expansion. So, if we want to expand $S(|x_j - x_i|)$ around $\bar{x}_C$, we can rewrite $x_j - x_i$ as $(x_j - \bar{x}_C) - (x_i - \bar{x}_C)$. Since we are expanding this with respect to x_j , $(\mathbf{x} - \mathbf{a})^{\mathbf{n}} = -(x_i - \bar{x}_C)$ and $f(\mathbf{a}) = S(x_j - \bar{x}_C)$

$$S(|x_j - x_i|) \approx \sum_{|\mathbf{n}| < p} \frac{(-1)^{|\mathbf{n}|}}{\mathbf{n}!} (x_i - \bar{x}_C)^{\mathbf{n}} \nabla^{\mathbf{n}} S(x_j - \bar{x}_C)$$

So now, subbing this into our expression for $\varphi(x_j)$

$$\varphi(x_j) \approx G \sum_{i \in \mathcal{C}} m_i \left[\sum_{|\mathbf{n}| < p} \frac{(-1)^{|\mathbf{n}|}}{\mathbf{n}!} (x_i - \bar{x}_C)^{\mathbf{n}} \nabla^{\mathbf{n}} S(x_j - \bar{x}_C) \right]$$

We can distribute the summations and write:

$$\varphi(x_j) \approx G \sum_{|\mathbf{n}| < p} \sum_{i \in \mathcal{C}} m_i \frac{(-1)^{|\mathbf{n}|}}{\mathbf{n}!} (x_i - \bar{x}_C)^{\mathbf{n}} \nabla^{\mathbf{n}} S(x_j - \bar{x}_C)$$

This first expression is called a *multi-pole moment* of the mass distribution in $\mathcal{C}$:

$$M_{\mathbf{n}}(\bar{x}_{\mathcal{C}}) = \sum_{i \in \mathcal{C}} m_i (x_i - \bar{x}_{\mathcal{C}})^{\mathbf{n}}$$

So then, we can write our multi-pole moment expansion as:

$$\varphi(x_j) \approx G \sum_{|\mathbf{n}| < p} \frac{(-1)^{|\mathbf{n}|}}{\mathbf{n}!} M_{\mathbf{n}}(\bar{x}_{\mathcal{C}}) \nabla^{\mathbf{n}} S(x_j - \bar{x}_{\mathcal{C}})$$

We can define the multi-pole moments recursively if we shift the evaluation point from $\bar{x}_{\mathcal{C}}$ to $\bar{x}_{\mathcal{C}} + x$, we can write:

$$M_{\mathbf{n}}(\bar{x}_{\mathcal{C}} + x) = \sum_{i \in \mathcal{C}} m_i (x_i - \bar{x}_{\mathcal{C}} - x)^{\mathbf{n}}$$

Using the binomial expansion, we can write:

$$(x_i - \bar{x}_{\mathcal{C}} - x)^{\mathbf{n}} = \sum_{\mathbf{k} \leq \mathbf{n}} (-1)^{|\mathbf{k}|} \binom{\mathbf{n}}{\mathbf{k}} (x_i - \bar{x}_{\mathcal{C}})^{\mathbf{n}-\mathbf{k}} x^{\mathbf{k}}$$

Substituting back, we get:

$$\begin{aligned} M_{\mathbf{n}}(\bar{x}_{\mathcal{C}} + x) &= \sum_{i \in \mathcal{C}} m_i \sum_{\mathbf{k} \leq \mathbf{n}} (-1)^{|\mathbf{k}|} \binom{\mathbf{n}}{\mathbf{k}} (x_i - \bar{x}_{\mathcal{C}})^{\mathbf{n}-\mathbf{k}} x^{\mathbf{k}} \\ &= \sum_{\mathbf{k} \leq \mathbf{n}} (-1)^{|\mathbf{k}|} \binom{\mathbf{n}}{\mathbf{k}} x^{\mathbf{k}} \sum_{i \in \mathcal{C}} m_i (x_i - \bar{x}_{\mathcal{C}})^{\mathbf{n}-\mathbf{k}} \\ &= \sum_{\mathbf{k} \leq \mathbf{n}} (-1)^{|\mathbf{k}|} \binom{\mathbf{n}}{\mathbf{k}} M_{\mathbf{n}-\mathbf{k}}(\bar{x}_{\mathcal{C}}) x^{\mathbf{k}} \end{aligned}$$

So then, to compute the multi-pole moments at the parent cell $\bar{x}_{\mathcal{P}}$, we have:

$$M_{\mathbf{n}}(\bar{x}_{\mathcal{P}}) = \sum_{\mathcal{C} \in \mathcal{P}} M_{\mathbf{n}}(\bar{x}_{\mathcal{C}} + (\bar{x}_{\mathcal{P}} - \bar{x}_{\mathcal{C}}))$$

Using the recursive formula, we get:

$$\begin{aligned} M_{\mathbf{n}}(\bar{x}_{\mathcal{P}}) &= \sum_{\mathcal{C} \in \mathcal{P}} \sum_{\mathbf{k} \leq \mathbf{n}} (-1)^{|\mathbf{k}|} \binom{\mathbf{n}}{\mathbf{k}} M_{\mathbf{n}-\mathbf{k}}(\bar{x}_{\mathcal{C}}) (\bar{x}_{\mathcal{P}} - \bar{x}_{\mathcal{C}})^{\mathbf{k}} \\ &= \sum_{\mathbf{k} \leq \mathbf{n}} (-1)^{|\mathbf{k}|} \binom{\mathbf{n}}{\mathbf{k}} \left(\sum_{\mathcal{C} \in \mathcal{P}} M_{\mathbf{n}-\mathbf{k}}(\bar{x}_{\mathcal{C}}) (\bar{x}_{\mathcal{P}} - \bar{x}_{\mathcal{C}})^{\mathbf{k}} \right) \end{aligned}$$

5 Fast Multipole Method: $O(n \log(n)) - O(n)$

We begin with the general expression for the potential at a point x_j due to a cell $\mathcal{C}$:

$$\varphi_{\mathcal{C}}(x_j) \approx -G \sum_{|\mathbf{n}| < p} \frac{(-1)^{|\mathbf{n}|}}{\mathbf{n}!} M_{\mathbf{n}}(\bar{x}_{\mathcal{C}}) \nabla^{\mathbf{n}} S(x_j - \bar{x}_{\mathcal{C}})$$

where:

- $M_n(\bar{x}_{\mathcal{C}})$ is the n -th multipole moment
- $S(r) = \frac{1}{r}$ is the point source kernel
- $\nabla^{\mathbf{n}}$ is the gradient operator corresponding to the multi-index $\mathbf{n}$

We expand the kernel $S(r)$ and compute up to the quadrupole term ($|n| \leq 2$).

Monopole Term ($n = 0$)

The monopole term is simply the total mass of the cell:

$$M_0(\bar{x}_{\mathcal{C}}) = \sum_{i \in \mathcal{C}} m_i$$

The corresponding potential is:

$$\varphi_{\text{monopole}}(x_j) = -\frac{GM_0(\bar{x}_{\mathcal{C}})}{r}$$

where $r = |x_j - \bar{x}_{\mathcal{C}}|$.

Given a the quad-tree structure, we can use data from it to calculate the mass of the cell, its center of mass, and therefore the monopole contribution to the potential due to cell $\mathcal{C}$:

$\varphi_{\mathcal{C}_{\text{monopole}}}(x_j)$ **Calculation**

- 1: **Input:** QuadTree $\mathcal{T}$, θ_{max} , Query Position x_j
- 2: **Output:** Monopole Potential at x_j due to cell $\mathcal{C}$
- 3: $x_{\min}, x_{\max}, y_{\min}, y_{\max}$, Particles $\leftarrow \mathcal{T}[\mathcal{C}]$
 $\triangleright$ Cell bounds returned from QuadTree structure.
- 4: $M_{\text{tot}} \leftarrow 0$ $\triangleright$ Initialize Total Mass of Cell
- 5: **for** p in Particles **do**
- 6: $M_{\text{tot}} \leftarrow M_{\text{tot}} + p.\text{mass}$
- 7: **end for**
- 8: $\bar{x}_{\mathcal{C}} \leftarrow 0$ $\triangleright$ Initialize Center of Mass
- 9: **for** p in Particles **do**
- 10: $\bar{x}_{\mathcal{C}} \leftarrow \bar{x}_{\mathcal{C}} + p.\text{Position} * p.\text{mass}$
- 11: **end for**
- 12: $\bar{x}_{\mathcal{C}} \leftarrow \frac{\bar{x}_{\mathcal{C}}}{M_{\text{tot}}}$
- 13: $M_0(\bar{x}_{\mathcal{C}}) \leftarrow M_{\text{tot}}$
return $-\frac{GM_0(\bar{x}_{\mathcal{C}})}{|x_j - \bar{x}_{\mathcal{C}}|}$

Quadrupole Term ($|n| = 2$)

The quadrupole moment is:

$$M_{kl}(\bar{x}_{\mathcal{C}}) = \sum_{i \in \mathcal{C}} m_i (x_{i,k} - \bar{x}_{\mathcal{C},k})(x_{i,l} - \bar{x}_{\mathcal{C},l})$$

Where $x_k = x_{j,k} - \bar{x}_{\mathcal{C},k}$, and the second derivative of the kernel is:

$$\nabla_k \nabla_l S(x_j - \bar{x}_{\mathcal{C}}) = \frac{1}{r^3} \left[\frac{3x_k x_l}{r^2} - \delta_{kl} \right]$$

Thus, the quadrupole contribution to the potential is:

$$\varphi_{\text{quadrupole}}(x_j) = -\frac{G}{2r^3} \sum_{k,l} M_{kl}(\bar{x}_{\mathcal{C}}) \left[\frac{3x_k x_l}{r^2} - \delta_{kl} \right]$$

We can note that $\sum_{k,l} M_{kl}(\bar{x}_{\mathcal{C}}) \delta_{kl} = \text{Tr}(M)$, and so:

$$\varphi_{\text{quadrupole}}(x_j) = -\frac{G}{2r^3} \left[\sum_{k,l} M_{kl}(\bar{x}_{\mathcal{C}}) \frac{3x_k x_l}{r^2} - \text{Tr}(M) \right]$$

$\varphi_{\mathcal{C}_{\text{quadrupole}}}(x_j)$ Calculation

```

1: Input: QuadTree  $\mathcal{T}$ ,  $\theta_{\max}$ , Query Position  $x_j$ 
2: Output: Quadrupole Potential at  $x_j$  due to cell  $\mathcal{C}$ 
3:  $x_{\min}, x_{\max}, y_{\min}, y_{\max}$ , Particles  $\leftarrow \mathcal{T}[\mathcal{C}]$  ▷ Cell bounds returned from QuadTree structure.
4:  $\bar{x}_{\mathcal{C}} \leftarrow 0$  ▷ Initialize Center of Mass
5: for  $p$  in Particles do
6:    $\bar{x}_{\mathcal{C}} \leftarrow \bar{x}_{\mathcal{C}} + p.\text{Position} * p.\text{mass}$ 
7: end for
8:  $\bar{x}_{\mathcal{C}} \leftarrow \frac{\bar{x}_{\mathcal{C}}}{M_{\text{tot}}}$ 
9:  $M \leftarrow []$  ▷ Initialize Quadrupole Tensor
10: for  $k$  in  $\text{len}(x_j)$  do
11:    $M[k] \leftarrow []$ 
12:   for  $l$  in  $\text{len}(x_j)$  do
13:      $M[k][l] \leftarrow 0$  ▷ Initialize Each Component at 0
14:     for  $p$  in Particles do
15:        $M[k][l] \leftarrow M[k][l] + p.\text{mass} * (p.\text{position}[k] - \bar{x}_{\mathcal{C}}[k])(p.\text{position}[l] - \bar{x}_{\mathcal{C}}[l])$ 
16:     end for
17:   end for
18: end for
19:  $\varphi(x_j) \leftarrow 0$ 
20: for  $l$  in  $\text{len}(x_j)$  do
21:   for  $k$  in  $\text{len}(x_j)$  do
22:      $\varphi(x_j) \leftarrow \varphi(x_j) + M[k][l] \frac{3 * \text{particles.position}[k] * \text{particles.position}[l]}{|x_j - \bar{x}_{\mathcal{C}}|^2}$ 
23:   end for
24: end for
25:  $\varphi(x_j) \leftarrow \varphi(x_j) - \text{Tr}(M)$ 
26:  $\varphi(x_j) \leftarrow -\frac{G}{2|x_j - \bar{x}_{\mathcal{C}}|^3} \varphi(x_j)$ 
    return  $\varphi(x_j)$ 

```

Combining the monopole, dipole, and quadrupole terms, we get the approximate potential:

$$\varphi_{\mathcal{C}}(x_j) \approx -\frac{G}{r} \left[M_0(\bar{x}_{\mathcal{C}}) + \frac{1}{2r^2} \left(\sum_{k,l} M_{kl}(\bar{x}_{\mathcal{C}}) \frac{3x_k x_l}{r^2} - \text{Tr}(M) \right) \right]$$

So, therefore to fully compute the potential at any point x_j , we have :

$\varphi(x_j)$ Calculation

```

1: Input: QuadTree  $\mathcal{T}$ ,  $\theta_{\max}$ , Query Position  $x_j$ 
2: Output: Potential at  $x_j$ 
3:  $\varphi(x_j) \leftarrow 0$ 
4: for  $\mathcal{C}$  in  $\mathcal{T}$  do
5:    $x_{\min}, x_{\max}, y_{\min}, y_{\max}, \text{Particles} \leftarrow \mathcal{T}[\mathcal{C}]$   $\triangleright$  Cell bounds returned from QuadTree structure.
6:    $S_{\mathcal{C}} \leftarrow \max(|x_{\min} - \bar{x}_{\mathcal{C}}|, |x_{\max} - \bar{x}_{\mathcal{C}}|, |y_{\min} - \bar{x}_{\mathcal{C}}|, |y_{\max} - \bar{x}_{\mathcal{C}}|)$ 
7:    $\theta_j \leftarrow \frac{S_{\mathcal{C}}}{|x_j - \bar{x}_{\mathcal{C}}|}$ 
8:   if  $\theta_j < \theta_{\max}$  then
9:      $M_{\text{tot}} \leftarrow 0$   $\triangleright$  Initialize Total Mass of Cell
10:    for  $p$  in Particles do
11:       $M_{\text{tot}} \leftarrow M_{\text{tot}} + p.\text{mass}$ 
12:    end for
13:     $\bar{x}_{\mathcal{C}} \leftarrow 0$   $\triangleright$  Initialize Center of Mass
14:    for  $p$  in Particles do
15:       $\bar{x}_{\mathcal{C}} \leftarrow \bar{x}_{\mathcal{C}} + p.\text{Position} * p.\text{mass}$ 
16:    end for
17:     $\bar{x}_{\mathcal{C}} \leftarrow \frac{\bar{x}_{\mathcal{C}}}{M_{\text{tot}}}$ 
18:     $M_0(\bar{x}_{\mathcal{C}}) \leftarrow M_{\text{tot}}$ 
19:     $\bar{x}_{\mathcal{C}} \leftarrow 0$   $\triangleright$  Initialize Center of Mass
20:    for  $p$  in Particles do
21:       $\bar{x}_{\mathcal{C}} \leftarrow \bar{x}_{\mathcal{C}} + p.\text{Position} * p.\text{mass}$ 
22:    end for
23:     $\bar{x}_{\mathcal{C}} \leftarrow \frac{\bar{x}_{\mathcal{C}}}{M_{\text{tot}}}$ 
24:     $M \leftarrow []$   $\triangleright$  Initialize Quadrupole Tensor
25:    for  $k$  in  $\text{len}(x_j)$  do
26:       $M[k] \leftarrow []$ 
27:      for  $l$  in  $\text{len}(x_j)$  do
28:         $M[k][l] \leftarrow 0$   $\triangleright$  Initialize Each Component at 0
29:        for  $p$  in Particles do
30:           $M[k][l] \leftarrow M[k][l] + p.\text{mass} * (p.\text{position}[k] - \bar{x}_{\mathcal{C}}[k]) * (p.\text{position}[l] - \bar{x}_{\mathcal{C}}[l])$ 
31:        end for
32:      end for
33:    end for
34:    for  $l$  in  $\text{len}(x_j)$  do
35:      for  $k$  in  $\text{len}(x_j)$  do
36:         $\varphi(x_j) \leftarrow \varphi(x_j) + M[k][l] \frac{3 * \text{particles.position}[k] * \text{particles.position}[l]}{|x_j - \bar{x}_{\mathcal{C}}|^2}$ 
37:      end for
38:    end for
39:     $\varphi(x_j) \leftarrow \varphi(x_j) - \text{Tr}(M)$ 
40:     $\varphi(x_j) \leftarrow -\frac{G}{2|x_j - \bar{x}_{\mathcal{C}}|^3} \varphi(x_j)$ 

```

```

41:      $\varphi(x_j) \leftarrow \varphi(x_j) - \frac{GM_{tot}}{|x_j - \bar{x}_C|}$ 
42:   else
43:     for  $p$  in Particles do
44:        $\varphi(x_j) \leftarrow \varphi(x_j) - \frac{G * p.mass}{|x_j - \bar{x}_C|}$ 
45:     end for
46:   end if
47: end for
   return  $\varphi(x_j)$ 

```

Now, in order to incorporate this into the Störmer-Verlet scheme, we will need to compute $-\nabla\varphi(x_j)$. The gradient of the potential $\varphi(x_j)$ is given by:

$$\nabla\varphi(x_j) = \frac{\partial\varphi(x_j)}{\partial x_{j,k}} \hat{\mathbf{e}}_k$$

where $\hat{\mathbf{e}}_k$ are the unit basis vectors in k -dimensions. We compute the gradient for each term (monopole, dipole, and quadrupole) as follows.

Gradient of the Monopole Term

The monopole term is:

$$\varphi_{\text{monopole}}(x_j) = -\frac{GM_0(\bar{x}_C)}{r}$$

where $r^2 = \sum_k (x_{j,k} - \bar{x}_{C,k})^2$ and $\mathbf{r} = \mathbf{x}_j - \bar{\mathbf{x}}_C$.

Taking the gradient:

$$\nabla\varphi_{\text{monopole}}(x_j) = \frac{\partial}{\partial x_{j,k}} \left(-\frac{GM_0(\bar{x}_C)}{r} \right) \hat{\mathbf{e}}_k$$

First, note that $\frac{\partial r}{\partial x_{j,k}} = \frac{r_k}{r}$, where $r_k = x_{j,k} - \bar{x}_{C,k}$. Thus:

$$\frac{\partial}{\partial x_{j,k}} \left(\frac{1}{r} \right) = -\frac{1}{r^2} \frac{\partial r}{\partial x_{j,k}} = -\frac{r_k}{r^3}$$

Therefore:

$$\frac{\partial\varphi_{\text{monopole}}(x_j)}{\partial x_{j,k}} = -GM_0(\bar{x}_C) \left(-\frac{r_k}{r^3} \right) = \frac{GM_0(\bar{x}_C)r_k}{r^3}$$

In vector form:

$$\nabla\varphi_{\text{monopole}}(x_j) = \frac{GM_0(\bar{x}_C)}{r^3} \mathbf{r}$$

$\mathbf{a}_{\text{monopole}}(x_j)$ Calculation

```

1: Input: QuadTree  $\mathcal{T}$ ,  $\theta_{\max}$ , Query Position  $x_j$ 
2: Output: Acceleration at  $x_j$  due to cell  $\mathcal{C}$ 
3:  $x_{\min}, x_{\max}, y_{\min}, y_{\max}$ , Particles  $\leftarrow \mathcal{T}[\mathcal{C}]$   $\triangleright$  Cell bounds returned from QuadTree structure.
4:  $M_{\text{tot}} \leftarrow 0$   $\triangleright$  Initialize Total Mass of Cell
5: for  $p$  in Particles do
6:    $M_{\text{tot}} \leftarrow M_{\text{tot}} + p.\text{mass}$ 
7: end for
8:  $\bar{x}_{\mathcal{C}} \leftarrow 0$   $\triangleright$  Initialize Center of Mass
9: for  $p$  in Particles do
10:    $\bar{x}_{\mathcal{C}} \leftarrow \bar{x}_{\mathcal{C}} + p.\text{Position} * p.\text{mass}$ 
11: end for
12:  $\bar{x}_{\mathcal{C}} \leftarrow \frac{\bar{x}_{\mathcal{C}}}{M_{\text{tot}}}$ 
13:  $M_0(\bar{x}_{\mathcal{C}}) \leftarrow M_{\text{tot}}$ 
    return  $-\frac{GM_0(\bar{x}_{\mathcal{C}})}{|x_j - \bar{x}_{\mathcal{C}}|^3}(x_j - \bar{x}_{\mathcal{C}})$ 

```

Gradient of the Quadrupole Term

The quadrupole term is:

$$\varphi_{\text{quadrupole}}(x_j) = -\frac{G}{2r^3} \left[\sum_{k,l} M_{kl}(\bar{x}_{\mathcal{C}}) \frac{3x_k x_l}{r^2} - \text{Tr}(M) \right]$$

Define:

$$W = \sum_{k,l} M_{kl}(\bar{x}_{\mathcal{C}}) x_k x_l$$

Then we can rewrite:

$$\varphi_{\text{quadrupole}}(x_j) = -\frac{G}{2} \left(\frac{3W}{r^5} - \frac{\text{Tr}(M)}{r^3} \right)$$

We need:

$$\nabla \varphi_{\text{quadrupole}}(x_j) = \frac{\partial \varphi_{\text{quadrupole}}}{\partial x_{j,m}} \hat{\mathbf{e}}_m$$

- Gradient of W : Since $W = \mathbf{x}^T M \mathbf{x}$ and M is symmetric:

$$\nabla W = 2M\mathbf{x}$$

- Gradient of r^{-3} :

$$\nabla(r^{-3}) = -3r^{-4} \nabla r = -3r^{-4} \frac{\mathbf{r}}{r} = -\frac{3\mathbf{r}}{r^5}$$

- Gradient of r^{-5} :

$$\nabla(r^{-5}) = -5r^{-6} \nabla r = -5r^{-6} \frac{\mathbf{r}}{r} = -\frac{5\mathbf{r}}{r^7}$$

- For the $\frac{3W}{r^5}$ term:

$$\nabla \left(\frac{3W}{r^5} \right) = 3 \left(\frac{W}{r^5} \right)' = \frac{3}{r^5} \nabla W + 3W \nabla \left(\frac{1}{r^5} \right)$$

Substitute $\nabla W = 2M\mathbf{x}$ and $\nabla \left(\frac{1}{r^5} \right) = -\frac{5\mathbf{r}}{r^7}$:

$$\nabla \left(\frac{3W}{r^5} \right) = \frac{3(2M\mathbf{x})}{r^5} + 3W \left(-\frac{5\mathbf{r}}{r^7} \right) = \frac{6M\mathbf{x}}{r^5} - \frac{15W\mathbf{r}}{r^7}$$

5. For the $\frac{\text{Tr}(M)}{r^3}$ term:

$$\nabla \left(\frac{\text{Tr}(M)}{r^3} \right) = \text{Tr}(M) \nabla(r^{-3}) = \text{Tr}(M) \left(-\frac{3\mathbf{r}}{r^5} \right) = -\frac{3 \text{Tr}(M)\mathbf{r}}{r^5}$$

Combine these results:

$$\nabla \varphi_{\text{quadrupole}}(x_j) = -\frac{G}{2} \left[\nabla \left(\frac{3W}{r^5} \right) - \nabla \left(\frac{\text{Tr}(M)}{r^3} \right) \right]$$

Substitute the gradients found:

$$\nabla \varphi_{\text{quadrupole}}(x_j) = -\frac{G}{2} \left[\left(\frac{6M\mathbf{x}}{r^5} - \frac{15W\mathbf{r}}{r^7} \right) - \left(-\frac{3 \text{Tr}(M)\mathbf{r}}{r^5} \right) \right]$$

$$\nabla \varphi_{\text{quadrupole}}(x_j) = -\frac{G}{2} \left[\frac{6M\mathbf{x}}{r^5} - \frac{15W\mathbf{r}}{r^7} + \frac{3 \text{Tr}(M)\mathbf{r}}{r^5} \right]$$

Where,

$$(M\mathbf{x})_m = \sum_k M_{mk} x_k$$

$$\mathbf{x}^T M \mathbf{x} = \sum_{k,l} x_k M_{kl} x_l$$

$$\text{Tr}(M) = \sum_k M_{kk}$$

$\mathbf{a}_{\text{quadrupole}}(x_j)$ Calculation

```

1: Input: QuadTree  $\mathcal{T}$ ,  $\theta_{\max}$ , Query Position  $x_j$ 
2: Output: Quadrupole Acceleration at  $x_j$  due to cell  $\mathcal{C}$ 
3:  $x_{\min}, x_{\max}, y_{\min}, y_{\max}$ , Particles  $\leftarrow \mathcal{T}[\mathcal{C}]$  ▷ Cell bounds returned from QuadTree structure.
4:  $\bar{x}_{\mathcal{C}} \leftarrow 0$  ▷ Initialize Center of Mass
5: for  $p$  in Particles do
6:    $\bar{x}_{\mathcal{C}} \leftarrow \bar{x}_{\mathcal{C}} + p.\text{Position} * p.\text{mass}$ 
7: end for
8:  $\bar{x}_{\mathcal{C}} \leftarrow \frac{\bar{x}_{\mathcal{C}}}{M_{\text{tot}}}$ 
9:  $M \leftarrow []$  ▷ Initialize Quadrupole Tensor
10: for  $k$  in  $\text{len}(x_j)$  do
11:    $M[k] \leftarrow []$ 
12:   for  $l$  in  $\text{len}(x_j)$  do
13:      $M[k][l] \leftarrow 0$  ▷ Initialize Each Component at 0
14:     for  $p$  in Particles do
15:        $M[k][l] \leftarrow M[k][l] + p.\text{mass} * (p.\text{position}[k] - \bar{x}_{\mathcal{C}}[k])(p.\text{position}[l] - \bar{x}_{\mathcal{C}}[l])$ 
16:     end for
17:   end for
18: end for
19:  $r \leftarrow x_j - \bar{x}_{\mathcal{C}}$ 
20:  $|r| \leftarrow \sqrt{\sum_m r[m]^2}$ 
21:  $W \leftarrow 0$ 
22: for  $k$  in  $\text{range}(\text{len}(x_j))$  do
23:   for  $l$  in  $\text{range}(\text{len}(x_j))$  do
24:      $W \leftarrow W + r[k] * M[k][l] * r[l]$ 
25:   end for
26: end for
27:  $\text{tr}M \leftarrow 0$ 
28: for  $k$  in  $\text{range}(\text{len}(x_j))$  do
29:    $\text{tr}M \leftarrow \text{tr}M + M[k][k]$ 
30: end for
31:  $Mx \leftarrow$  zero vector of size  $\text{len}(x_j)$ 
32: for  $m$  in  $\text{range}(\text{len}(x_j))$  do
33:   for  $k$  in  $\text{range}(\text{len}(x_j))$  do
34:      $Mx[m] \leftarrow Mx[m] + M[m][k] * r[k]$ 
35:   end for
36: end for
37:  $a \leftarrow$  zero vector of size  $\text{len}(x_j)$ 
38: for  $m$  in  $\text{range}(\text{len}(x_j))$  do
39:    $a[m] \leftarrow \frac{G}{2} \left( \frac{6Mx[m]}{|r|^5} - \frac{15W * r[m]}{|r|^7} + \frac{3\text{tr}M * r[m]}{|r|^5} \right)$ 
40: end for

```

So, for the full acceleration, we can write:

```

                                a( $x_j$ ) Calculation
1: Input: QuadTree  $\mathcal{T}$ ,  $\theta_{\max}$ , Query Position  $x_j$ 
2: Output: Acceleration at  $x_j$ 
3:  $\mathbf{a} \leftarrow$  zero vector of size  $\dim(x_j)$ 
4: for  $\mathcal{C}$  in  $\mathcal{T}$  do
5:   ( $x_{\min}, x_{\max}, y_{\min}, y_{\max}$ , Particles)  $\leftarrow \mathcal{T}[\mathcal{C}]$ 
6:    $M_{tot} \leftarrow 0$ 
7:   for  $p$  in Particles do
8:      $M_{tot} \leftarrow M_{tot} + p.\text{mass}$ 
9:   end for
10:   $\bar{x}_{\mathcal{C}} \leftarrow 0$ 
11:  for  $p$  in Particles do
12:     $\bar{x}_{\mathcal{C}} \leftarrow \bar{x}_{\mathcal{C}} + p.\text{Position} * p.\text{mass}$ 
13:  end for
14:   $\bar{x}_{\mathcal{C}} \leftarrow \frac{\bar{x}_{\mathcal{C}}}{M_{tot}}$ 
                                      $\triangleright$  Compute size  $S_{\mathcal{C}}$  of cell
15:   $S_{\mathcal{C}} \leftarrow \max(|x_{\min} - \bar{x}_{\mathcal{C}}|, |x_{\max} - \bar{x}_{\mathcal{C}}|, |y_{\min} - \bar{x}_{\mathcal{C}}|, |y_{\max} - \bar{x}_{\mathcal{C}}|)$ 
16:   $r \leftarrow x_j - \bar{x}_{\mathcal{C}}$ 
17:   $|r| \leftarrow \sqrt{\sum_m r[m]^2}$ 
18:   $\theta_j \leftarrow \frac{S_{\mathcal{C}}}{|x_j - \bar{x}_{\mathcal{C}}|}$ 
19:  if  $\theta_j < \theta_{\max}$  then
                                      $\triangleright$  Monopole and Quadrupole Approximation
                                      $\triangleright$  Compute Quadrupole Tensor M
20:     $M \leftarrow$  zero matrix of size  $\dim \times \dim$ 
21:    for  $p$  in Particles do
22:       $\delta \leftarrow p.\text{Position} - \bar{x}_{\mathcal{C}}$ 
23:      for  $k$  in  $\text{range}(\dim(x_j))$  do
24:        for  $l$  in  $\text{range}(\dim(x_j))$  do
25:           $M[k][l] \leftarrow M[k][l] + p.\text{mass} * \delta[k] * \delta[l]$ 
26:        end for
27:      end for
28:    end for
                                      $\triangleright$  Compute W, trM, and Mx
29:     $W \leftarrow 0$ 
30:    for  $k$  in  $\text{range}(\dim(x_j))$  do
31:      for  $l$  in  $\text{range}(\dim(x_j))$  do
32:         $W \leftarrow W + r[k] * M[k][l] * r[l]$ 
33:      end for
34:    end for
35:     $\text{tr}M \leftarrow 0$ 
36:    for  $k$  in  $\text{range}(\dim(x_j))$  do
37:       $\text{tr}M \leftarrow \text{tr}M + M[k][k]$ 
38:    end for
39:     $Mx \leftarrow$  zero vector
40:    for  $m$  in  $\text{range}(\dim(x_j))$  do

```

```

41:         for  $k$  in range(dim( $x_j$ )) do
42:              $Mx[m] \leftarrow Mx[m] + M[m][k] * r[k]$ 
43:         end for
44:     end for
45:                                      $\triangleright$  Monopole Acceleration
46:      $a_{\text{monopole}} \leftarrow -\frac{GM_{\text{tot}}}{|r|^3} r$ 
47:                                      $\triangleright$  Quadrupole Acceleration
48:     for  $m$  in range(dim( $x_j$ )) do
49:          $a_{\text{quad}}[m] = \frac{G}{2} \left( \frac{6Mx[m]}{|r|^5} - \frac{15Wr[m]}{|r|^7} + \frac{3\text{tr}(M)r[m]}{|r|^5} \right)$ 
50:     end for
51:                                      $\triangleright$  Total Acceleration from Cell
52:      $a \leftarrow a + a_{\text{monopole}} + a_{\text{quad}}$ 
53: else                                      $\triangleright$  Direct Summation of Particle Accelerations
54:     for  $p$  in Particles do
55:          $\Delta r \leftarrow x_j - p.\text{Position}$ 
56:          $\Delta|r| \leftarrow |\Delta r|$ 
57:          $a \leftarrow a - \frac{Gp.\text{mass}}{\Delta|r|^3} \Delta r$ 
58:     end for
59: end if
60: end for
61: return  $a$ 

```

6 Multipole-QuadTree N-Body Simulation

So, now we have the full expression for the potential and acceleration, and we can build the full simulation. The simulation structure will take the following form:

Störmer-Verlet Algorithm with Multipole Potential

```

1: Input: Initial positions  $\mathbf{q}[0]$ , initial momenta  $\mathbf{p}[0]$ , timestep  $h$ , number of
   steps  $N_{\text{steps}}$ , QuadTree  $\mathcal{T}$ ,  $\theta_{\text{max}}$ 
2: Output: Arrays  $\mathbf{q}$ ,  $\mathbf{p}$  with positions and momenta over time
3:                                      $\triangleright$  Build the initial QuadTree
4:  $\mathcal{T} \leftarrow \text{BuildQuadTree}(\mathbf{q}[0], \text{masses}, \text{parameters})$ 
5: for  $n = 0$  to  $N_{\text{steps}} - 1$  do                                      $\triangleright$  Outer loop for time steps
6:                                      $\triangleright$  Half-step momentum update using  $\varphi_{\text{multipole}}$ 
7:   for  $i = 1$  to  $N_{\text{particles}}$  do                                      $\triangleright$  Inner loop for particles
8:      $\mathbf{p}_{i,n+\frac{1}{2}} \leftarrow \mathbf{p}_{i,n} + \frac{h}{2} m_i \mathbf{a}(\mathbf{q}_{i,n}, \mathcal{T}, \theta_{\text{max}})$ 
9:   end for
10:                                      $\triangleright$  Full-step position update
11:  for  $i = 1$  to  $N_{\text{particles}}$  do                                      $\triangleright$  Inner loop for particles
12:     $\mathbf{q}_{i,n+1} \leftarrow \mathbf{q}_{i,n} + h \frac{\mathbf{p}_{i,n+\frac{1}{2}}}{m_i}$ 
13:  end for
14:                                      $\triangleright$  Rebuild the QuadTree for new positions
15:   $\mathcal{T} \leftarrow \text{BuildQuadTree}(\mathbf{q}[n+1], \text{masses}, \text{parameters})$ 
16:                                      $\triangleright$  Final half-step momentum update
17:  for  $i = 1$  to  $N_{\text{particles}}$  do                                      $\triangleright$  Inner loop for particles
18:     $\mathbf{p}_{i,n+1} \leftarrow \mathbf{p}_{i,n+\frac{1}{2}} + \frac{h}{2} m_i \mathbf{a}(\mathbf{q}_{i,n+1}, \mathcal{T}, \theta_{\text{max}})$ 
19:  end for
20: end for
21: return  $\mathbf{q}, \mathbf{p}$ 

```

So, implementing this structure with 200 orbiting bodies and two central masses, we get the following simulated system. Although close encounters do lead to unrealistically high velocities for some of the particles, the overall behavior of the system is consistent with the expected behavior, and is vastly more efficient than the direct summation method employed at the beginning of the document. To resolve these high velocities, in the future, I will employ a model which assigns radii to the bodies by assigning them a random density of $2 \text{ g/cm}^3 \leq \rho \leq 5 \text{ g/cm}^3$, then using their randomly assigned mass to calculate some radius for the bodies (assuming a uniformly distributed mass), and perform the calculations for inelastic collisions between bodies. This, however, is beyond the scope of this project, and currently collisions are not resolved in the simulation. Despite the lack of collision resolution in this simulation, it still demonstrates the effectiveness of tree structures for n-body simulations, and showcases how series approximations are used in physics to improve the efficiency of calculations when such approximations are appropriate. Source files for this project can be accessed at [this github repository](#).

References

- [1] Leslie Greengard and Vladimir Rokhlin. A fast algorithm for particle simulations. *Journal of Computational Physics*, 73(2):325–348, 1987.
- [2] Josh Barnes and Piet Hut. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nature*, 324(6096):446–449, 1986.
- [3] Walter Dehnen. A very fast and momentum-conserving tree code. *The Astrophysical Journal Letters*, 536(1):L39–L42, 2000.
- [4] James Binney and Scott Tremaine. *Galactic Dynamics*. Princeton University Press, Princeton, NJ, 2 edition, 2008.
- [5] R. W. Hockney and J. W. Eastwood. *Computer Simulation Using Particles*. CRC Press, Bristol, UK, 1988.
- [6] Charles F. Gammie and Frederic H. Rasio. *Computational Astrophysics*. University of Illinois Press, Urbana, IL, 2016.
- [7] Jo Bovy. Dynamics and astrophysics of galaxies, 2023. Available at <https://galaxiesbook.org/>.
- [8] Walter Dehnen and Justin I. Read. N-body simulations of gravitational dynamics. *European Physical Journal Plus*, 129(2):1–28, 2014.
- [9] Volker Springel. The cosmological simulation code gadget-2. *Monthly Notices of the Royal Astronomical Society*, 364(4):1105–1134, 2005.